

```
CREATE [OR REPLACE] PROCEDURE procedure_name
[(param_name datatype [IN | OUT | IN OUT] [, ...])]
{IS | AS}
BEGIN
    -- Logic here
EXCEPTION
    -- Handle exceptions here
END [procedure_name];
```

```
CREATE [OR REPLACE] FUNCTION function_name
[(param_name datatype [, ...])]
RETURN return_datatype {IS | AS}
BEGIN
    -- Logic here
EXCEPTION
    -- Handle exceptions here
END [function_name];
```

```
BEGIN
    -- Example logic
IF :NEW.column_name IS NULL THEN
    RAISE_APPLICATION_ERROR(-20001, 'Column cannot be null');
END IF;
EXCEPTION
    WHEN OTHERS THEN
        -- Handle unexpected exceptions
        DBMS_OUTPUT.PUT_LINE('Error occurred: ' || SQLERRM);
END;
```

```
IF condition THEN
    -- Logic
ELSEIF another_condition THEN
    -- Logic
ELSE
    -- Logic
END IF;
```

1. Anchors

- **^**: Start of string.
- **\$**: End of string.

2. Quantifiers

- *****: 0 or more.
- **+**: 1 or more.
- **?**: 0 or 1.
- **{n}**: Exactly **n**.
- **{n, }**: At least **n**.
- **{n, m}**: Between **n** and **m**.

3. Character Classes

- **[abc]**: Any of **a**, **b**, **c**.
- **[^abc]**: Not **a**, **b**, **c**.
- **[a-z]**: Lowercase letters.
- **[A-Z]**: Uppercase letters.
- **[0-9]**: Digits.
- **.**: Any character (except newline);

```
DECLARE
    CURSOR cursor_name IS
        SELECT column_name [, ...] FROM table_name WHERE condition;

BEGIN
    OPEN cursor_name;
    LOOP
        FETCH cursor_name INTO variable [, ...];
        EXIT WHEN cursor_name%NOTFOUND;
        -- Logic here
    END LOOP;
    CLOSE cursor_name;
EXCEPTION
    -- Handle exceptions here
END;
```

```
CREATE [OR REPLACE] TRIGGER trigger_name
{BEFORE | AFTER | INSTEAD OF} [INSERT OR | UPDATE OR | DELETE]
ON table_name
[FOR EACH ROW]
DECLARE
    -- Optional declarations
BEGIN
    -- Logic here
EXCEPTION
    -- Handle exceptions here
END;
```

```
FOR i IN 1..10 LOOP
    -- Logic
END LOOP;

WHILE condition LOOP
    -- Logic
END LOOP;

LOOP
    -- Logic
    EXIT WHEN condition;
END LOOP;
```

```
CREATE SEQUENCE sequence_1
start with 1
increment by 1
minvalue 0
maxvalue 100
cycle;
```

```
SELECT first_name, last_name
FROM employees
WHERE REGEXP_LIKE (first_name, '^Ste(v|ph)en$');
```

```
CREATE TABLE Departments (
    DepartmentID INT PRIMARY KEY,
    DepartmentName VARCHAR(50) NOT NULL
);

-- Create a child table with a foreign key reference
CREATE TABLE Employees (
    EmployeeID INT PRIMARY KEY,
    EmployeeName VARCHAR(50) NOT NULL,
    DepartmentID INT,
    FOREIGN KEY (DepartmentID)
        REFERENCES Departments(DepartmentID)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);
```

```
DROP TABLE ENROLLMENTS;
DROP TABLE STUDENT;
DROP TABLE COURSE;
```

```
CREATE TABLE student (
    Name VARCHAR(16),
    DOB DATE,
    studID INTEGER PRIMARY KEY NOT NULL
);

CREATE TABLE course (
    courseID INTEGER PRIMARY KEY NOT NULL,
    Name VARCHAR2(16)
);

CREATE TABLE enrollments (
    enrollID INTEGER PRIMARY KEY NOT NULL,
    courseID INTEGER,
    studID INTEGER,
    FOREIGN KEY (courseID) REFERENCES course(courseID),
    FOREIGN KEY (studID) REFERENCES student(studID)
);
```

```
IF NOT EXISTS (SELECT 1 FROM professors WHERE professorID = professorID) THEN  
    RAISE_APPLICATION_ERROR(-20001, 'Invalid professor ID');  
END IF;
```

```
EXCEPTION  
    WHEN OTHERS THEN  
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
```

```
FOREIGN KEY (column_name) REFERENCES other_table(column_name);
```